Guía Práctica Nº 4 - Programación Competitiva 2012

1) Mostrar que:
   a) $50\ n - 8$ pertenece a $O(n)$
   b) $5n^2 - 3n + 6$ pertenece a $O(n^2)$
   c) $2^n + 7n + 3$ pertenece a $O(2^n)$

2) Determinar el orden y omega de las siguientes funciones:
   a) $5n^2$
   b) $3n^2 + 2n$
   c) $3^n + 2^n$
   d) $n + log_2(n)$
   e) $n^2 + log_2(n)$

3) Determinar el orden zeta de las siguientes funciones:
   a) $n^2 + 2n$

   b) $t(n) = \begin{cases} n+4, \text{ si } n \text{ es par} \\ 7n^2 + n, \text{ si } n \text{ es impar.} \end{cases}$

   c) $t(n) = \begin{cases} n+4, \text{ si } n \text{ es par} \\ 5n + n, \text{ si } n \text{ es impar} \end{cases}$

4) Reescriba los siguientes órdenes de complejidad de menor a mayor:
   O(n), O(n log n), O(n!), O(n^2), O(5), O(log log n), O(3^n), O(sqrt(n)), O(n^n), O(n^3), O(2^n), O(log n),

5) Para cada uno de los seis fragmentos de código siguientes:
   - Diga que orden de complejidad poseen:
   - Implementelos y de el tiempo de ejecución para diferentes valores de n
   - Comparar estimación teórica con tiempos reales.

a)
```
sum = 0
for i = 1...n do
        sum = sum + 1
end
```

b)
```
sum = 0
for i = 1...n do
        for j = 1...n do
                sum = sum + 1
        end
end
```

c)

```
sum = 0
for i = 1...n do
        for j = 1...n^2 do
                sum = sum + 1
        end
end
```

d)

```
sum = 0
for i = 1...n do
        for i = 1...i do
                sum = sum + 1
        end
end
```

e)

```
sum = 0
for i = 1...n do
        for i = 1...i^2 do
                for k=1 to j do
                        sum = sum + 1
                end
        end
end
```

f)

```
i = 1
  while ( i <= n)
        j = 1
        while ( j <= n)
                j = j * 2
        end
        i = i + 1
  end
```

b)

```
  i = 1
  while ( i <= n)
        j = 1
        while ( j <= i)
                j = j +1
        end
        i = i + 1
  end
```

c)

```
i = 1
while ( i <= 10)
        j = 1
        while ( j <= 10)
                j = j + 1
        end
        i = i + 2
end
```

6) ¿Cuál es el propósito de los siguientes algoritmos? Analizar el tiempo de ejecución en el peor caso y expresarlos en notación O.

a)

```
long f1 (int a, int n) {
        long b = 1;
        for (int k =0; k < n; k++= {
                b = b*a;
        }
        return b;
}
```

b)

```
long f2 (int a, int n) {
        long b = 1;
        long c = a;

        for (int k=n; k>0; ) {
                if (k%2 == ) {
                        k = k/2;
                        c = c*c;
                }
                else {
                        k--;
                        b = b*c;
                }
        }
        return b;
}
```

7) A partir de una secuencia de números A, calcular otra B tal que cada uno de sus elementos B[i] sea el promedio de todos los anteriores en la secuencia original (incluído i).
Ejemplo:
A = [1, 3, 5, 4, 2, 1]
B= [1, 2, 3, 3.25, 3, 2.66]

8) Dada una secuencia de enteros, posiblemente negativos, encontrar e identificar la subsecuencia correspondiente al valor máximo de la sumatoria desde Ai hasta Aj. Cuando todos los valores son negativos, dar secuencia vacía y suma 0.
Evaluar complejidad.

9) Un arreglo A contiene n-1 enteros únicos en el intervalo [0, n-1]; esto es hay un número de este intervalo que no está en A. Diseñar un algoritmo para encontrar ese número en tiempo O(n). Solo se permite usar O(1) espacio adicional además del arreglo A mismo.

Ejemplo:
- Sea N= 5
- El arreglo se puede completar con valores entre [0;(N-1)] , [0;4]
- El arreglo tiene n-1 elementos: 4
  - vector= [1, 4, 2, 0] → "El valor que falta es 3"

10) Un elemento mayoritario en un vector A, de tamaño N, es un elemento que aparece más de N/2 veces (en consecuencia habrá como máximo un elemento mayoritario, pudiendo no haber).
Por ejemplo:
- {*3*,*3*,**4**,*2*,**4,4**,*2*,**4**,**4**}
4 es el elemento mayoritario (5 apariciones > 9/2 = V)
- {*3*,*3*,**4**,*2*,**4,4**,*2*,**4**}
No hay elemento mayoritario (4 apariciones de 4 > 8/2 = F)
Escriba un algoritmo para encontrar el elemento mayoritario cuando exista, o que indique que no existe. Indique cual es el tiempo de ejecución del algoritmo que construyó.

11) Suponga que cada renglón de un arreglo A de ***NxN*** posee valores 1 y 0 de tal modo que en cualquier renglón de A todos los 1 van antes de que los 0.
Describir un método con tiempo de ejecución O(n) (*y no* ***O(n²)*** *)* para determinar el renglón de A que contiene la mayor cantidad de unos.

Ejemplo

1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	0	0	0	0
1	1	1	0	0

12) El algoritmo A ejecuta 10 n log n operaciones, mientras que el algoritmo B ejecuta n^2 operaciones.
a) Cuál de los dos algoritmos posee un índice de crecimiento mayor?
b) A partir de qué valor A siempre ejecuta menos operaciones que B?

13) Un algoritmo tarda 0.5 milisegundos para una entrada de tamaño 100. Determinar cuanto tardará con una entrada de tamaño 500 si el tiempo de ejecución es:
a) lineal
b) O (n log n)
c) Cuadrático
d) Cúbico

14) - La computadora TroncoPC, muy utilizada desde hace unos años, puede ejecutar 10000 operaciones básicas en un minuto. La nueva computadora FastPC puede ejecutar 1000000 de operaciones básicas en un minuto! (es 100 veces más rápida).

a) Un algoritmo con función de tiempo f(n) = 10 n, ejecutando sobre TroncoPC, procesa un tamaño máximo de datos de entrada n=1000, en un minuto.

Cuál será el nuevo tamaño n' (cantidad máxima de datos), que puede procesar el mismo algoritmo, en la computadora FastPC, en un minuto?

b) Qué sucede si el algoritmo tiene función de tiempo f(n) = n^2 ?

c) Qué sucede si el algoritmo tiene función de tiempo f(n) = n^3 ?

d) Qué sucede si el algoritmo tiene función de tiempo f(n) = 2^n ?

15) Resolver los siguientes problemas, y para cada uno de ellos determinar el orden de complejidad de su solución:

a) Eights: http://www.spoj.pl/problems/EIGHTS.pdf

b) Light, more light: http://acm.uva.es/p/v101/10110.html

c) Fctrl: http://www.spoj.pl/problems/FCTRL/